AI-Assisted Guest Support Assistant Design Document

Cassidie Grogan, Ezra Begashaw, Benjamin Dulcio, Kendal Elison, Wilfred Faltz

Spring 2026

This form is completed in accordance with the Software Engineering Body of Knowledge (SWEBOK).

1. Introduction

1.1 Purpose

The purpose of this document is to present the design and architectural for a Proof-of-Concept or PoC AI-assisted guest support chat platform intended for a high-volume vacation rental business. The proposed system aims to reduce the call-center workload by automating responses to automating responses to routine guest inquires using real-time property and reservation data. The platform will integrate a Large Language Model or LLM with verified operational data retrieved through the StreamlineVRS API to generate contextually accurate responses. Emphasis will be placed on prompt engineering techniques to constrain model outputs, reduce hallucination risk, and ensure that generated responses remain grounded in authoritative data sources. This document defines the system's objectives, scope, architecture, reference materials, and stakeholder requirements.

1.2 Scope

This project focuses on the development of a functional Proof-of-Concept system capable of handling common guest inquiries such as check-in/check-out times, reservation details, cancelation policies, property amenities and other routine informational requests. The system will consist of a lightweight web-based conversational interface, a Python-based backend service, and integration with a large language model provider. The backend will retrieve structured data through the StreamlineVRS API and inject validated information into model prompts to ensure response accuracy.

The scope of this PoC is limited to demonstrating technical feasibility and validating the effectiveness of AI-assisted automation for routine guest communication. Production-grade infrastructure, advanced analytics, multilingual capabilities, voice interaction, and deep integration with additional enterprise systems are outside the scope of this initial implementation.

1.3 Overview

High-volume vacation rental organizations frequently encounter substantial operational demands due to repetitive guest inquiries. Many of these inquiries involve standardized information that can be programmatically retrieved from property management systems. The proposed solution introduces a conversational AI assistant capable of interfacing with the StreamlineVRS API to obtain real-time reservation and property data. This data will be incorporated into structured prompts submitted to a large language model, which will generate natural language responses for end users through a web-based chat interface.

The system architecture will follow a modular design, separating concerns between the frontend interface, backend application logic, API integration layer, and LLM service. Special consideration will be given to retrieval-grounded response generation, structured prompt design, and confidence-based escalation mechanisms to ensure that complex or ambiguous inquiries are redirected to human agents. Logging and observability mechanisms will be implemented to evaluate model performance, response quality, and hallucination frequency during the PoC evaluation phase.

1.4 Definitions and Acronyms:

Acronym	Meaning
AI	Artificial Intelligence
UI	User Interface
LLM	Large Language Model
POC	Proof of Concept
SRS	Software Requirements Specification

Term	Definition
StreamlineVRS	A property management system used by Vacations for YOU to manage reservations and property information
Guest	An individual with an existing reservation at a Vacations for YOU property
Call Deflection	Reduction in customer support calls through automated self-service solutions
Retrieval	Augmented Generation (RAG) is a design pattern in which external data is retrieved and incorporated into prompts to improve factual grounding. Prompt Engineering refers to the systematic design of model inputs to control output quality, tone, and factual constraints.

Table 1. Definitions

1.5 References

[1] Institute of Electrical and Electronics Engineers, IEEE Std 1016-2017 – IEEE Standard for Information Technology, Systems Design, Software Design Descriptions, New York, NY, USA: IEEE, 2017.

[2] OpenAI, “OpenAI API Documentation,” 2026. [Online]. Available: https://platform.openai.com/docs [Accessed: Feb. 18, 2026].

[3] StreamlineVRS, “StreamlineVRS API Documentation,” 2026. [Online]. Available: https://developer.streamlinevrs.com [Accessed: Feb. 18, 2026].

[4] Requirements Documentation. “Requirements Specification for an AI-Assisted Guest Support Assistant,” 2026. [Documentation] [Accessed Feburary 18, 2026].

2. System Overview

2.1 Architectural Design

The AI-assisted guest support chat platform will follow a modular layered, client-server architecture. This separation of components improves maintainability and scalability while remaining appropriate for the POC implementation. The architecture of the platform consists of a lightweight web-based frontend using React, Python-based backend service using FastAPI, an AI processing layer utilizing a Large Language Model (LLM), and an integration layer responsible for secure communication with the StreamlineVRS API.

The front-end provides the conversational UI through which guests interact, submit questions, and receive responses. Communication to the back end will be done through HTTPS REST Endpoints. All integration with the StreamlineVRS API is strictly read-only to prevent reservation data from being modified by the system. The backend service will manage session handling and reservation validation. The AI processing component will implement a Retrieval-Augmented Generation (RAG) pattern, through which relevant reservation and property data are retrieved from StreamlineVRS prior to constructing the prompts. These prompts will then be submitted to an external LLM provider, such as OpenAI, to generate NL responses relevant to the guest’s inquiries.

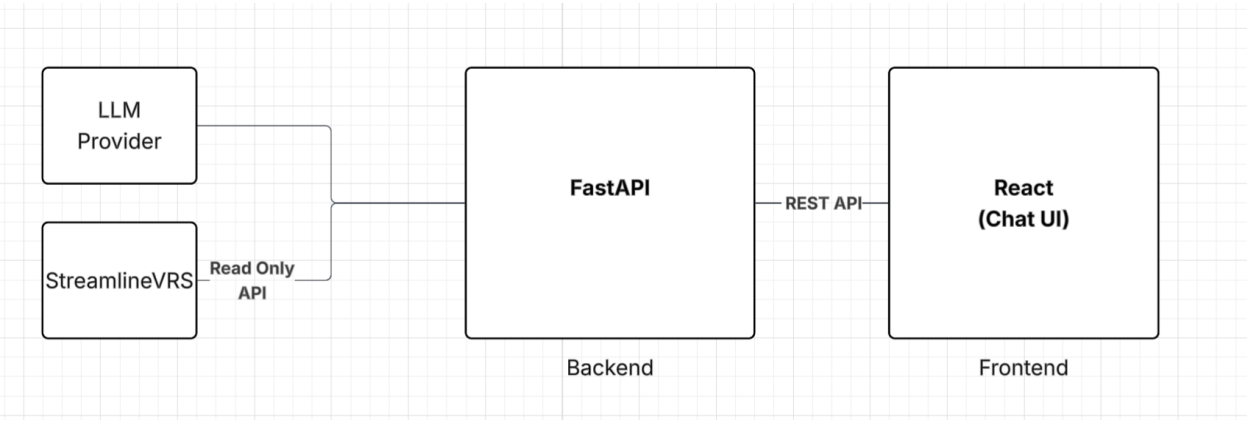


Figure 1. High Level Architecture Design for AI Assistant Chat Bot

2.2 Technical Design

The frontend consists of a browser-based chat interface intended to capture guest input and display system responses in a clean, readable format. The backend is implemented using a Python-based web service exposing RESTful endpoints. Core backend responsibilities include confirming reservation information against the StreamlineVRS API, retrieving relevant operational data, constructing structured prompts, invoking the LLM API, evaluating response confidence, and logging interactions.

Upon successful reservation validation, a session identifier is generated and used to maintain conversational state. When a guest submits an inquiry, the backend retrieves the relevant reservation/property data from the API and injects this information into the prompt. The prompt explicitly constraints the LLM to use only the supplied data when generating a response. The structured prompt is then submitted to the LLM provider.

After receiving the model output, the backend applies for validation and confidence checks. If the response meets the predefined criteria, it is then returned to the frontend. Else if the question is outside defined system scope or fails the criteria, the system provides escalation instructions directly to guest to human support.

2.3 Design Rationale

A modular layered architecture was selected to clearly separate user interfaces from backend logic, AI processing, and API integration. This design reduces coupling between components, improves readability, simplifies testing, and allows scalability beyond POC.

The RAG approach was chosen to address one of the primary risks associated with LLMs and hallucination. Retrieving authentic reservation and property data from StreamlineVRS prior to response generation ensures that the outputs are grounded in verified information. Read-only integration with StreamlineVRS was enforced to minimize security risk and data manipulation.

2.4 Architectural Views

The system is logically divided into four layers:

Presentation Layer

A chatbot based interface that guests can interact with to submit natural language questions to receive accurate and human-like responses. This layer is responsible only for user interaction and does not contain business logic.

Application Layer

A web service, predominantly based in Python, responsible for session management, request handling, prompt construction, response validation, hallucination control, and escalation when required. This layer contains the core business logic of the system.

Integration Layer

This layer abstracts communication with external APIs, including:

- The Large Language Model (LLM) provider (initially OpenAI API)
- The future reservation/property management system API (e.g., StreamlineVRS)

The integration layer isolates third-party dependencies, allowing provider substitution without architectural redesign.

AI processing Layer

The AI processing component leverages a hosted LLM via the OpenAI API to generate natural language responses. During Phase 1 of the PoC, the LLM will operate using structured prompt constraints and predefined business rules.

2.5 Architectural Assumptions

- For the POC, API integration is not finalized
- OpenAI API will serve as the initial LLM provider with the option to switch over when needed
- Financial and payment inquiries are excluded as well as the ability to alter reservations.

3. Detailed Design

3.1 Decomposition Description

The system is decomposed through the four architectural layers defined in Section 2.4. The Presentation Layer consists of the React based chat interface responsible for rendering conversational elements and capturing guest input. The Application Layer, implemented using FastAPI, contains the core business logic, which includes session management, reservation validation, structured prompt construction, response confidence evaluation, and escalation handling. This layer also manages interaction logging and observability to support evaluation of model performance and hallucination frequency during the PoC.

The Integration Layer abstracts all communication with external services, including the StreamlineVRS API for reservation and property data retrieval, and the OpenAI API for LLM processing. As noted in Section 2.5, API integration with StreamlineVRS is not finalized for the Phase 1 PoC, so this layer is designed to accommodate mock data sources during initial development. The AI Processing Layer handles the submission of structured prompts to the LLM and operates under predefined business rules and prompt constraints that limit the model to responding only with information grounded in the supplied data.

3.2 Dependency Description

Module dependencies follow a unidirectional flow. The Presentation Layer depends on the Application Layer to validate reservation information before starting a guest session. The Application Layer depends on the Integration Layer to retrieve reservation and property data from the StreamlineVRS API, which is then used to construct structured prompts. The Application Layer also depends on the AI Processing Layer to submit assembled prompts to the OpenAI API and return generated responses.

All external dependencies are isolated within the Integration Layer, which allows the LLM provider or property management of API to be substituted without impacting the Application or Presentation layers. The Presentation Layer has no direct dependency on external services and communicates exclusively with the Application Layer through HTTPS REST endpoints. This separation prevents changes to third-party APIs from affecting the entire system

3.3 Interface Description

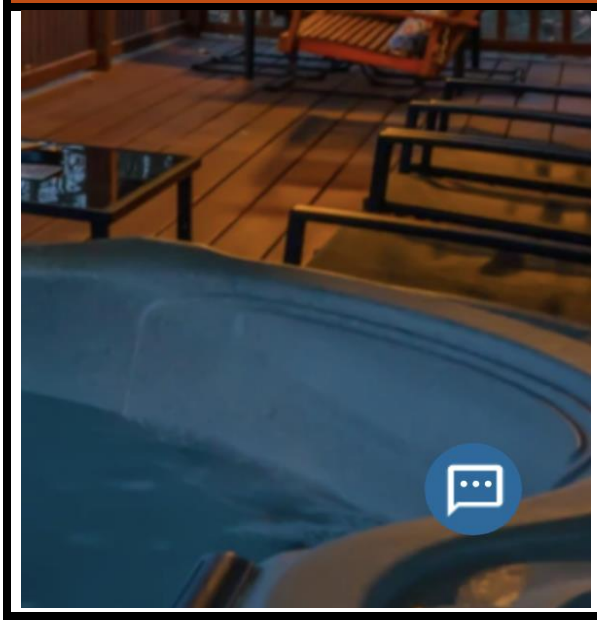
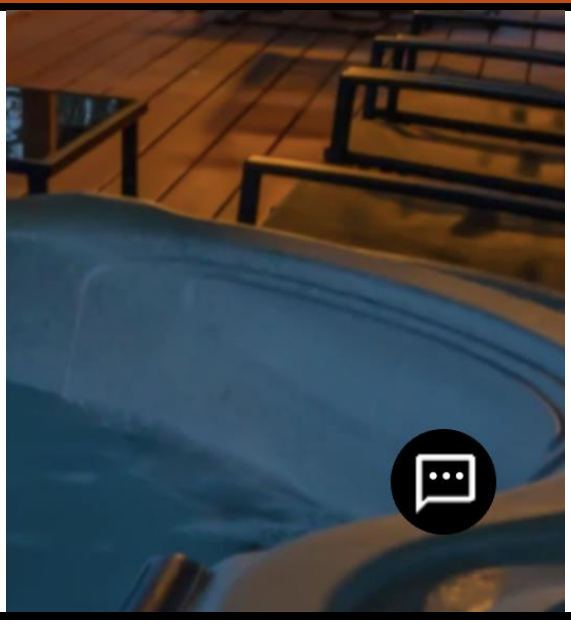
The React frontend communicates with the Fast API backend exclusively through HTTPS REST endpoints. A reservation validation endpoint accepts guest reservation information and returns a session token upon successful verification. A message endpoint accepts the guest's natural language inquiry, and the session token and returns the AI-generated response, or an escalation notice directing the guest to human support. All endpoints require a valid session token to prevent unauthorized access to reservation data.

Internally, the backend communicates with the StreamlineVRS API through read-only RESTful requests to retrieve reservation and property data, ensuring that no external records are modified by the system. Communication with the OpenAI API follows the standard chat completions contract, submitting structured message arrays and receiving generated text responses. Error handling is applied at each interface boundary so that API failures or timeouts result in graceful shutdowns and appropriate logging rather than unhandled system errors.

4. User Interface Design

The Phase 1 user interface will be a web-based, fully conversational chatbot interface designed to provide a simple and intuitive guest support experience. The interface will resemble modern messaging platforms, allowing guests to enter questions in natural language and receive conversational responses in real time.

4.1 UI Design

Option 1	Option 2
	

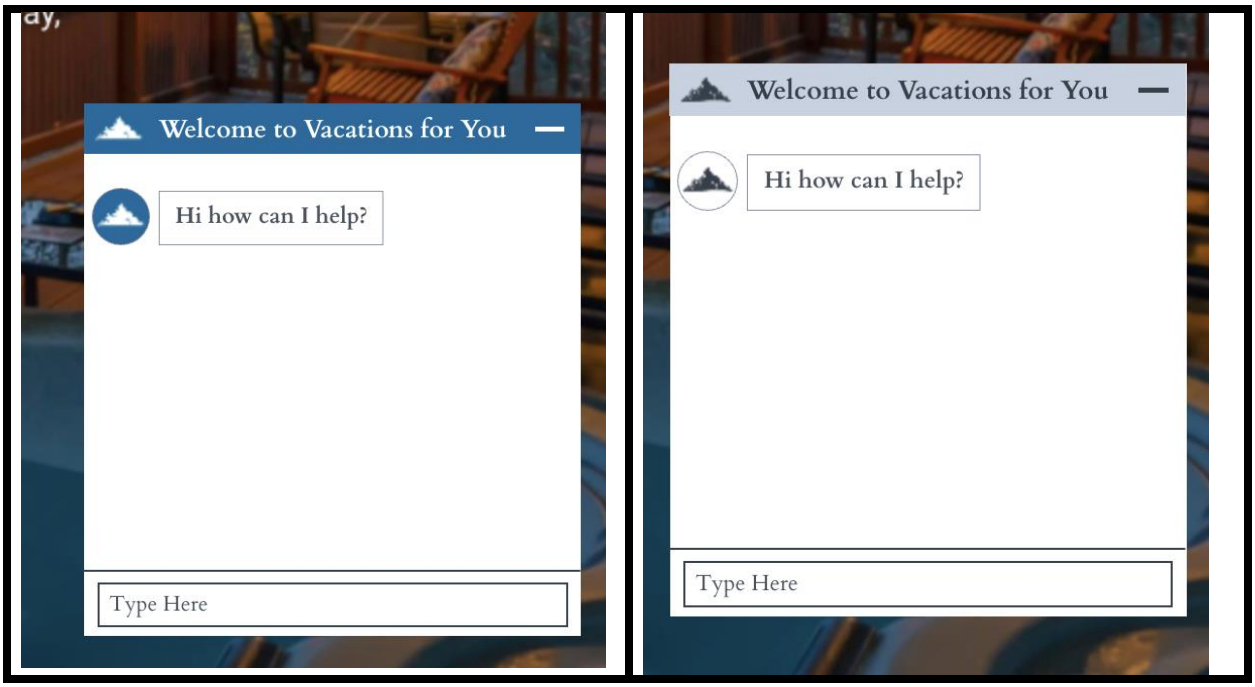


Table 2. User Interface Designs

5. Database and Data Design

The data flow within the system follows a linear, request-driven model. User-generated input originates at the frontend interface and is transmitted to the backend application server. The backend acts as the central orchestration point, processing the request and determining whether the inquiry falls within supported categories.

For Phase 1 of development, the backend will either:

- Reference predefined structured reservation data stored internally for simulation purposes, or
- Operate without live reservation retrieval, focusing solely on constrained informational responses.

The backend constructs a structured prompt containing system instructions, business constraints, and any relevant contextual data. This prompt is transmitted to the OpenAI API. The LLM processes the prompt and generates a natural language response.

The backend receives the response, performs validation and scope checks, and then transmits the finalized response back to the frontend for presentation to the user.

At no point does the client communicate directly with the OpenAI API or any external reservation system. All external communication occurs exclusively within the backend layer to preserve architectural integrity and security control.

6. API Integration Plan

6.1 Overview

The system will expose a RESTful API to handle user queries and coordinate interactions between the frontend interface and the LLM service.

The API will act as middleware responsible for:

- Receiving user questions
- Retrieving structured reservation or property data
- Formatting contextual prompts
- Sending requests to the LLM
- Returning validated responses to the user

6.1 Core Integration – Conceptual

Figure 2 illustrates the backend request processing flow for the AI-powered reservation assistant. The process begins when a user submits a question through the front-end interface. The backend API first validates the request to ensure that required parameters are present and properly formatted. If validation fails, an error response is returned immediately. If the request is valid, the system determines whether reservation or property data is required to generate an accurate response. When necessary, the backend retrieves relevant structured data before constructing a contextualized prompt.

This prompt is then transmitted to the OpenAI API for natural language processing. Upon receiving the model response, the backend performs validation and formatting before returning a finalized answer to the user. This flow ensures structured processing, controlled decision-making, and reliable integration with external services.

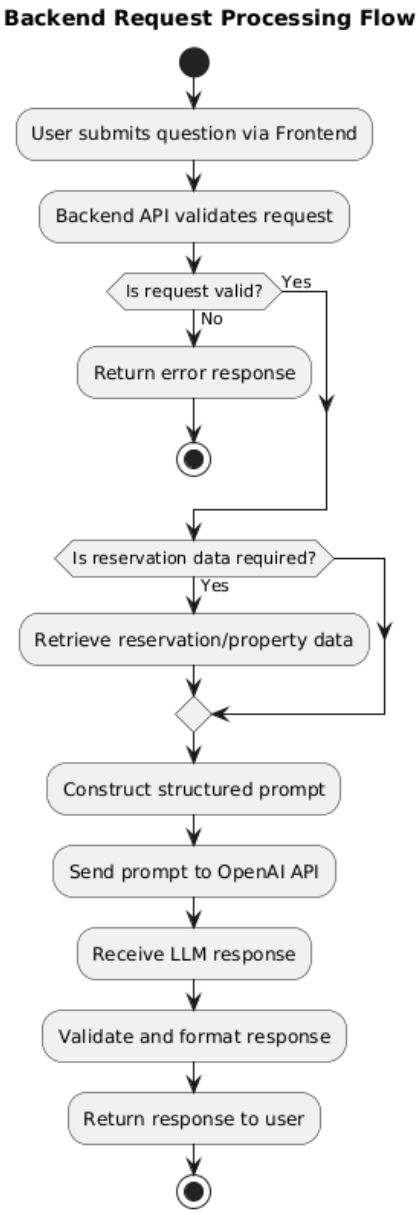


Figure 2. Activity Diagram demonstrating User Interaction with the Chatbot Interface

Figure 3 presents the sequence of interactions between system components during query processing. The interaction begins when the user submits a question through the frontend interface, which sends a POST request to the backend API. The backend validates the request and conditionally retrieves reservation data if required.

After assembling a structured prompt, the backend sends a request to the OpenAI API for language model processing. The OpenAI API returns a generated response, which the backend formats and relays to the frontend for display to the user.

The following sequence diagram highlights the order of communication between internal components and external services, clearly demonstrating the system's reliance on third-party AI processing while maintaining backend control over validation and response formatting.

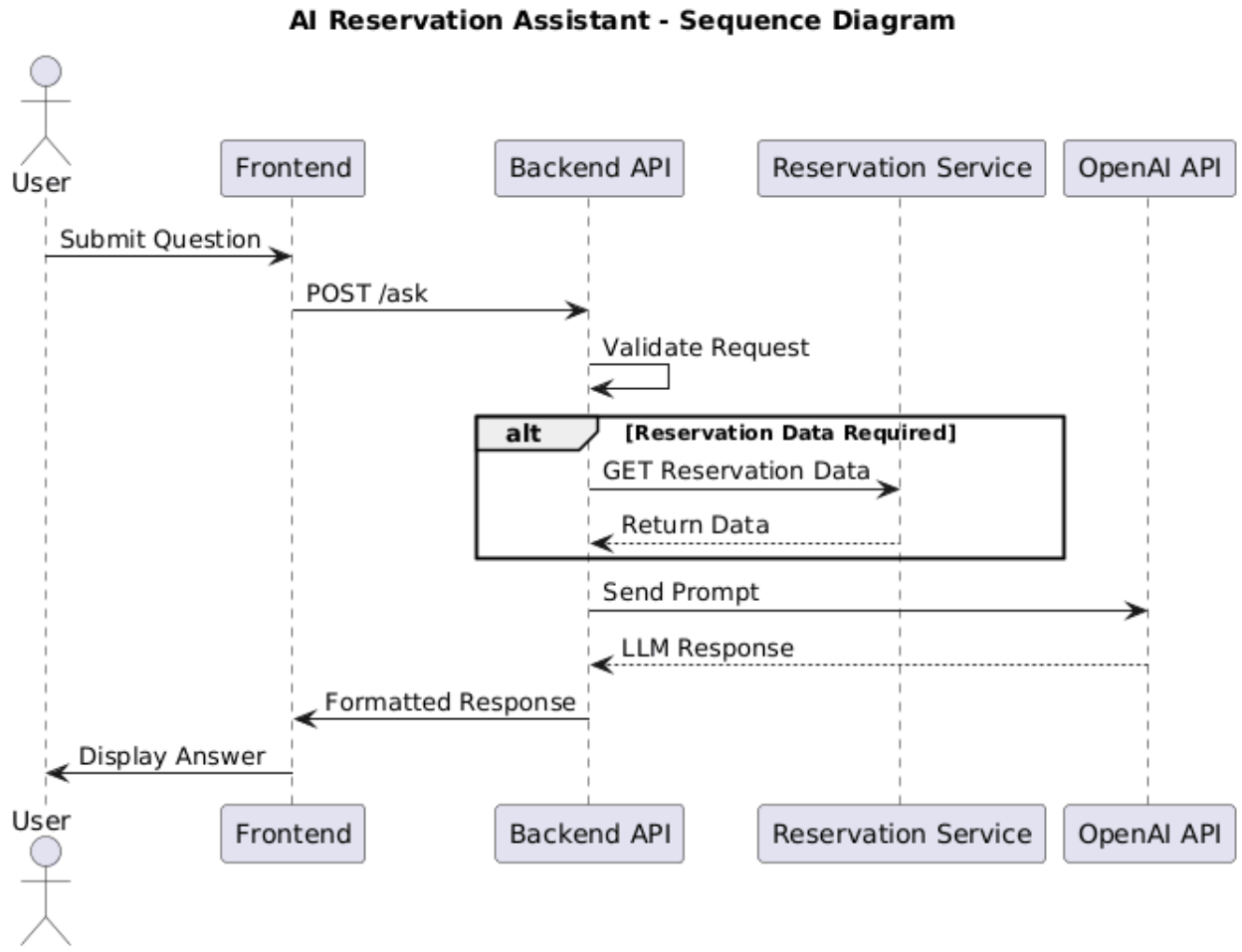


Figure 3. Sequence Diagram demonstrating the interaction between the system components

6.2 Evaluation Metrics

The system will be evaluated using quantitative and qualitative performance measures. Accuracy will be measured by comparing AI-generated responses against verified reservation and property data. Response time will be measured as the duration between user submission and system reply. Reliability will be evaluated through repeated testing of

identical queries to ensure consistent output. Usability will be assessed through user feedback surveys and task completion of success rates.

7. Appendices

Figure 1. High-Level Architecture Design for the AI Assistant Chatbot

Figure 2. Activity Diagram Demonstrating User Interaction with the Chatbot Interface

Figure 3. Sequence Diagram Demonstrating Interaction Between System Components

Table 1. Definitions

Table 2. User Interface Designs

8. Change Log

Date	Version	Changes
2/18/26	1.0	Initial Design Document